

## A. BLOCKLY/SCRATCH/FLOWCHART

This week focused on getting familiar with the basics of computing through visual programming tools, flowcharts, and core logic concepts.

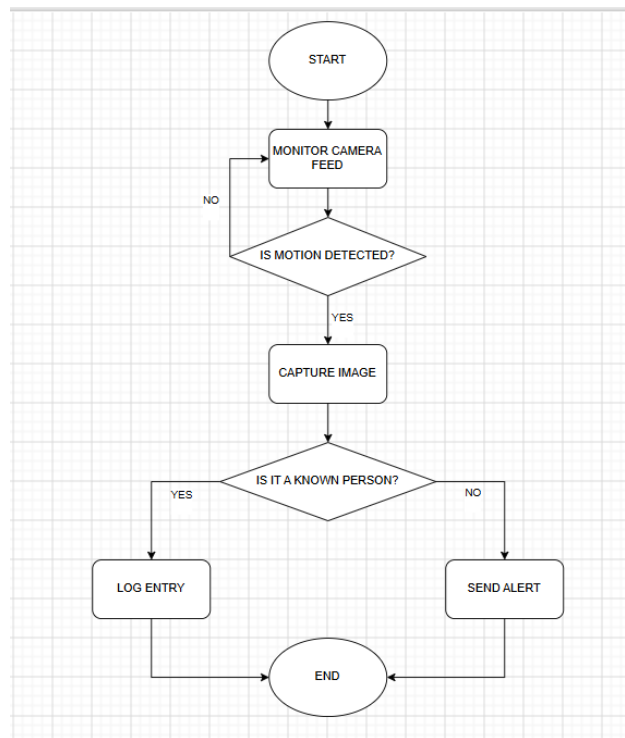
Using Scratch, I worked on building simple interactive projects where sprites respond to events like key presses and mouse clicks. This introduced sequencing, events, and how programs are structured around triggers and responses.

Blockly was used to work through logic puzzles and maze challenges. The block-based format made it easy to understand loops and conditions practically. For example, using a repeat block instead of placing the same instruction multiple times, or using an if-else block to handle different outcomes.

Alongside these tools, I covered flowcharts as a way to plan and visualise decision-making before writing any code. Mapping out a process step by step, including what happens when a condition is true or false helped in understanding how programs follow logical paths.

### Core Concepts Covered

- **Variables** are used to store and update values within a program. A simple example is a score counter in a Scratch game that increases each time the player earns a point. The variable holds the current value and updates it as the program runs.
- **Loops** allow a set of instructions to repeat without writing them out multiple times. In Blockly, instead of placing the same movement block ten times, a repeat loop handles it in one step. This makes code shorter, cleaner, and easier to manage.
- **Events** are triggers that cause specific parts of a program to run. In Scratch, clicking the green flag, pressing a key, or a sprite touching another object are all events that set off different actions. Programs are built around listening for these triggers and responding to them.
- **Conditions** use if-else logic to allow a program to make decisions. Depending on whether something is true or false, the program takes a different path. For example, if the score reaches a certain number, play a sound otherwise, keep going. This is what allows programs to behave differently based on what is happening at any given moment.



**CAMERA ALERT FLOWCHART**

## B. PYTHON LIBRARIES

### 1. NumPy

Full name: Numerical Python

Import convention: `import numpy as np`

NumPy is the foundation of numerical computing in Python. It is used for working with large, multi-dimensional arrays and matrices of numerical data. We can think of it as a supercharged calculator built for scientific computations.

What it does:

- Stores data in arrays (like lists, but far more powerful and faster)
- Performs mathematical operations: addition, multiplication, finding the mean, and much more across entire datasets at once
- Works efficiently even with very large amounts of data

Example:

python

```
a = np.array([1, 2, 3])
```

This code defines a NumPy array containing the values 1, 2, and 3. From here we can perform operations on the entire array in a single line.

---

### 2. Pandas

Import convention: `import pandas as pd`

Pandas is a library built for data manipulation and analysis. It makes it easy to work with data stored in tables called DataFrames allowing us to clean, organize, filter, and analyze structured data.

A common environment to work with Pandas is Jupyter Notebook, a popular open-source platform that lets us write and run code interactively, cell by cell.

What it does:

- Reads data from external files (CSV, Excel, JSON, etc.)
- Lets you filter rows, sort columns, handle missing values, and merge tables
- Makes exploratory data analysis fast and intuitive

Example:

```
python
```

```
pd.read_csv("data.csv")
```

This code reads a CSV file and loads it into a DataFrame, ready for analysis.

---

### 3. Matplotlib

Import convention: `import matplotlib.pyplot as plt`

Matplotlib is a library for creating static, interactive, and dynamic visual representations of data. It is the most widely used plotting library in Python and gives us full control over how our charts look.

What it does:

- Plots bar charts, histograms, scatter plots, line graphs, pie charts, and more
- Allows customization of colors, labels, titles, axes, and styles
- Serves as the base layer that many other visualization libraries are built on top of

Example: Visualizing how sales change over time using a line graph: each point on the line represents sales at a given time, making trends easy to spot.

---

### 4. Seaborn

Import convention: `import seaborn as sns`

Seaborn is built on top of Matplotlib as an extension designed specifically for statistical data visualization. It produces more attractive and informative graphs with less code, making it ideal for exploring relationships in data.

What it does:

- Creates heatmaps, pair plots, violin plots, box plots, and distribution plots
- Automatically handles a lot of the aesthetic styling that would require manual work in Matplotlib
- Works directly with Pandas DataFrames, making integration seamless

Example: Comparing the relationships between multiple variables in a dataset. For instance, seeing how age, income, and spending score relate to each other across customer groups using a pair plot.

---

## 5. Scikit-learn

Import convention: from sklearn import ...

Scikit-learn is Python's go-to library for implementing machine learning algorithms. It provides simple and efficient tools to build, train, evaluate, and improve models all without needing to code the algorithms from scratch.

What it does:

- Implements algorithms for regression, classification, and clustering
- Includes tools to split data into training and testing sets
- Provides model evaluation metrics to measure how well your model performs
- Offers data preprocessing tools like scaling, encoding, and transforming raw data before feeding it into a model

Example: Building a model to predict house prices based on features like size, number of rooms, and location Scikit-learn handles everything from splitting the data to training the model to evaluating its accuracy.

## C. FASTAPI

---

### Core Concepts

#### Endpoint

A specific URL in your API that performs an action. For example, /get-student is one endpoint, / is another. Each endpoint is mapped to a function that runs when that URL is accessed.

#### GET

Used to fetch or read data. When you visit a URL in your browser, you are making a GET request. No data is sent in the body. You are simply asking the server for something back.

```
from fastapi import FastAPI

app = FastAPI()

students = {
    1: {
        "name": "john",
        "age": 17,
        "class": "year 12"
    }
}

@app.get("/")
def index():
    return {"name": "First Data"}

@app.get("/get-student/{student_id}")
def get_student(student_id: int):
    return students[student_id]
```

```
@app.get("/")
def index():
    return {"name": "First Data"}

@app.get("/get-student/{student_id}")
def get_student(student_id: int = Path(None, description="The ID of the student you want to view")):
    return students[student_id]
```

## POST

Used to send or create new data. For example, submitting a form to create a new student. You send data to the server in the request body and it processes or saves it.

```
@app.post("/create-student/{student_id}")
def create_student(student_id: int, student: Student):
    if student_id in students:
        return {"Error": "Student exists"}

    students[student_id] = student
    return students[student_id]
```

## PUT

Used to update existing data. For example, if a student changes their class, you send a PUT request to update that specific record on the server.

```
@app.put("/update-student/{student_id}")
def update_student(student_id: int, student: UpdateStudent):
    if student_id not in students:
        return {"Error": "Student does not exist"}

    if student.name != None:
        students[student_id].name = student.name

    if student.age != None:
        students[student_id].age = student.age

    if student.year != None:
        students[student_id].year = student.year

    return students[student_id]
```

## DELETE

Used to remove data. A /delete-user or /delete-student endpoint accepts a DELETE request along with an ID and removes that record from the system.

```
@app.delete("/delete-student/{student_id}")
def delete_student(student_id: int):
    if student_id not in students:
        return {"Error": "Student does not exist"}

    del students[student_id]
    return {"Message": "Student deleted successfully"}
```

## Create-User / Delete-User Endpoints

These are named endpoints which perform certain actions:

- /create-user → receives a POST request along with user information and creates a user object
  - /delete-user → receives a DELETE request along with the ID of a user and deletes that user object
- 

## JSON Data

Standard data format used by APIs. Looks similar to a Python dictionary:

json

```
{"name": "john", "age": 17}
```

JSON is everything that goes back and forth between your application and who calls your API: requests and responses.

---

## Home Endpoint (/)

Endpoint / is the homepage of your API. It is the first thing being hit when accessing the API through its base URL. Usually, returns a welcoming message or some other simple information about the API.

---

## Documentation in FastAPI (localhost/docs)

In FastAPI, an interactive documentation is automatically created for you in /docs. With that, you can:

- View all your endpoints in one place
- Interact with them directly in your browser
- Discover the information the endpoint is supposed to send and receive

This documentation is automatically created by FastAPI using your code.

---

## Path Parameter

This is where there is a variable directly placed within the URL path. For instance:

```
/get-student/{student_id}
```

{student\_id} will vary depending on the student you are fetching. If you visit /get-student/1 you get student 1 while visiting /get-student/2 gives you student 2. The path parameter is the dynamic part and it serves as the key used to retrieve data.

---

## Query Parameter

It's a variable attached to the end of a URL after a ? For instance:

```
/get-student?name=john
```

Unlike the path parameters, the query parameters are optional and they are mostly used in filtering or searching or for additional purposes without altering the base URL of the endpoint.

---

## Path() in FastAPI

When you add Path() to your path parameters you have two main advantages:

1. Description → A human-readable description that automatically appears in your /docs page
2. Validation rules → You can limit the input to be only certain types of values

Example:

python

```
student_id: int = Path(None, description="The ID of the student you want to view")
```

---

### Validation Rules: gt / lt / ge / le

These are comparison-based validation constraints passed inside Path() or Query() to control what values are accepted:

| Rule | Meaning                  | Example | Effect                                                    |
|------|--------------------------|---------|-----------------------------------------------------------|
| gt   | Greater than             | gt=0    | Value must be more than 0 so 0 is rejected, 1 passes      |
| lt   | Less than                | lt=100  | Value must be below 100 so 100 is rejected, 99 passes     |
| ge   | Greater than or equal to | ge=1    | Value must be 1 or above so 1 passes, 0 is rejected       |
| le   | Less than or equal to    | le=100  | Value must be 100 or below so 100 passes, 101 is rejected |

These rules reject invalid input before your logic even runs, keeping your API clean and safe.

---

### Combining Path and Query Parameters

Both can be used together in a single endpoint. The path parameter comes from the URL structure, while the query parameter is appended after ?. FastAPI handles both automatically based on how you define your function parameters.

```
@app.get("/get-by-name/{student_id}")
def get_student(*, student_id: int, name: Optional[str] = None, test : int):
    for student_id in students:
        if students[student_id]["name"] == name:
            return students[student_id]
    return {"Data": "Not found"}
```

---

## D. OPENCV

### A) FUNCTIONS

#### 1. Image to Grayscale

Grayscale conversion eliminates all color information from the picture and retains only the intensity of the pixels (level of brightness). The process makes computations easier because the information about every pixel becomes one value instead of three (BGR).

#### 2. Blur (Gaussian Blur)

Blurring is required to get rid of noise in an image before further manipulations. Gaussian blur calculates the weighted average of neighboring pixels with higher importance assigned to the ones in the central area.

#### 3. Edge Detection (Canny)

Edge detection finds the borders of objects in the picture. Canny algorithm detects edges based on gradient values using hysteresis thresholding to retain only relevant edges.

#### 4. Dilation

Dilation uses structuring element called kernel to expand bright areas in the picture. When applied to Canny edges, it enlarges them. Useful for closing gaps and connecting edges.

#### 5. Erosion

Erosion is basically the reverse process of dilation; it reduces the size of white regions by eliminating pixels from the boundary. When the same parameters (kernel size and number of iterations) are used in dilation, erosion can almost restore the edge condition of the original image. Good for reducing small white noise or separating adjacent images.

#### 6. Resize

Image resize refers to changing the dimensions (height and width) of the image. This can either be enlargement or reduction of the size of the image. OpenCV cv2.resize function allows one to specify exact size or scaling factor of the image.

#### 7. Cropping

The cropping technique involves the selection of an image region of interest (ROI) by using array slicing with numpy. For instance, `image[y1:y2, x1:x2]`.

```
# Converting to grayscale
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
cv.imshow('Gray', gray)

# Blur
blur = cv.GaussianBlur(img, (7,7), cv.BORDER_DEFAULT)
cv.imshow('Blur', blur)

# Edge Cascade
canny = cv.Canny(blur, 125, 175)
cv.imshow('Canny Edges', canny)
```

```
# Dilating the image
dilated = cv.dilate(canny, (7,7), iterations=3)
cv.imshow('Dilated', dilated)

# Eroding
eroded = cv.erode(dilated, (7,7), iterations=3)
cv.imshow('Eroded', eroded)
```



```
# Resize
resized = cv.resize(img, (500,500), interpolation=cv.INTER_CUBIC)
cv.imshow('Resized', resized)

# Cropping
cropped = img[50:200, 200:400]
cv.imshow('Cropped', cropped)
```

## B) IMAGE TRANSFORMATIONS

### 1. Translation

Translation moves the image either up, down, left, or right, a certain number of pixels. It uses a transformation matrix and is accomplished using `cv2.warpAffine()`.

```
import cv2 as cv
import numpy as np

img = cv.imread('Photos/boston.jpg')
cv.imshow('Boston', img)

# Translation
def translate(img, x, y):
    transMat = np.float32([[1,0,x],[0,1,y]])
    dimensions = (img.shape[1], img.shape[0])
    return cv.warpAffine(img, transMat, dimensions)

# -x --> Left
# -y --> Up
# x --> Right
# y --> Down

translated = translate(img, 100, 100)
cv.imshow('Translated', translated)

cv.waitKey(0)
```

### 2. Rotation

Rotation makes the image rotate about a point at a certain angle. OpenCV has the function `cv2.getRotationMatrix2D()` which generates the matrix for rotation and then the same matrix is used by `cv2.warpAffine()`. Rotation allows you to manipulate the scaling as well.

```
def rotate(img, angle, rotPoint=None):
    (height,width) = img.shape[:2]

    if rotPoint is None:
        rotPoint = (width//2,height//2)

    rotMat = cv.getRotationMatrix2D(rotPoint, angle, 1.0)
    dimensions = (width,height)

    return cv.warpAffine(img, rotMat, dimensions)
```

### 3. Flipping

The flipping operation flips the image in an axis:

- Vertical flip – 0 (inverts the image from bottom to top)
- Horizontal flip – 1 (flips the image horizontally)

- BOTH vertical and horizontal flipping – -1

```
# Flipping
flip = cv.flip(img, 0)
cv.imshow('Flipped', flip)
```

## C) CONTOUR DETECTION

**1. Finding Contours** The `cv2.findContours()` function gives two outputs – contours and hierarchies.

- The number of contours is the number of elements in the contours list.
- The number of edges detected in an image is reduced by blurring the image before passing it into the Canny Function.

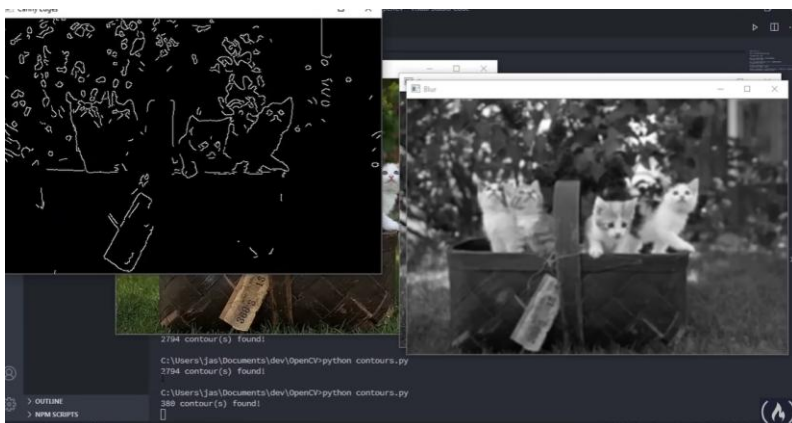
```
cv.imshow('Cats', img)

gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
cv.imshow('Gray', gray)

blur = cv.GaussianBlur(gray, (5,5), cv.BORDER_DEFAULT)
cv.imshow('Blur', blur)

canny = cv.Canny(blur, 125, 175)
cv.imshow('Canny Edges', canny)

contours, hierarchies = cv.findContours(canny, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)
print(f'{len(contours)} contour(s) found!')
```



## 2. Threshold Method (Alternative to Canny)

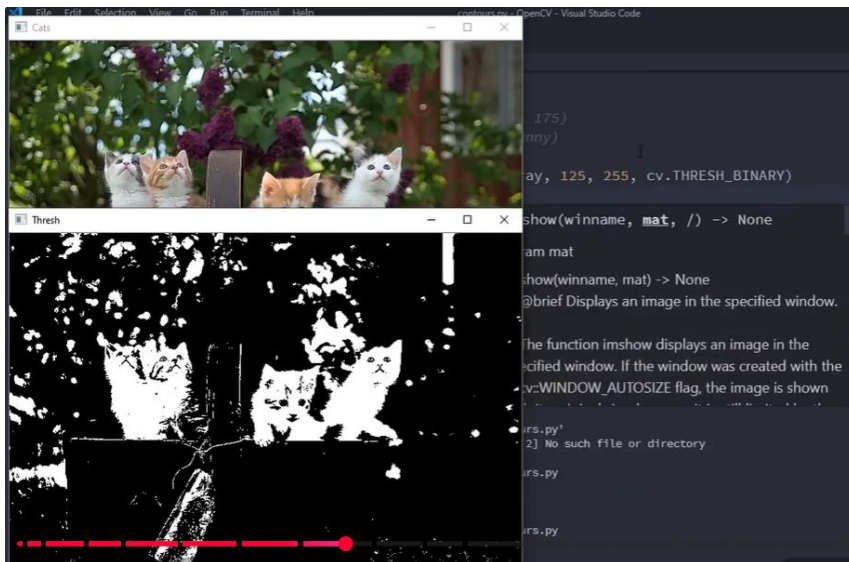
Canny edge detector is not used here; instead, the image is binarized through thresholding.

- If pixel value < x → Pixel is black (0)
- If pixel value > x → Pixel is white (255)

This results in a crisp black and white image that can now be used to detect contours.

```
ret, thresh = cv.threshold(gray, 125, 255, cv.THRESH_BINARY)

contours, hierarchies = cv.findContours(thresh, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)
print(f'{len(contours)} contour(s) found!')
```



### 3. Drawing Contours on a Blank Image

After finding contours, you can visualize them using `cv2.drawContours()` on a blank (black) image of the same size. This lets you see only the outlines of detected shapes, isolated from the original image.

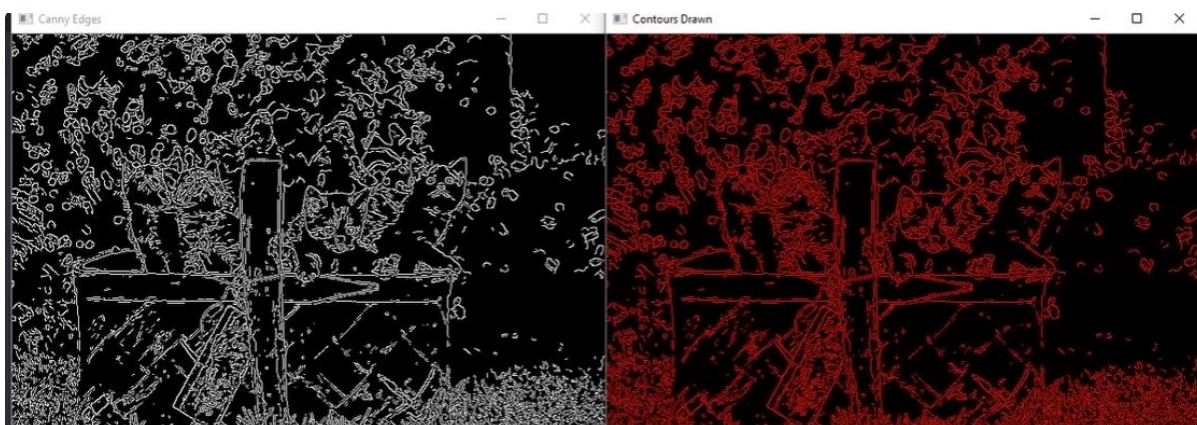
```
cv.imshow('Canny Edges', canny)

# ret, thresh = cv.threshold(gray, 125, 255, cv.THRESH_BINARY)
# cv.imshow('Thresh', thresh)

contours, hierarchies = cv.findContours(canny, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)
print(f'{len(contours)} contour(s) found!')

cv.drawContours(blank, contours, -1, (0,0,255), 1)
cv.imshow('Contours Drawn', blank)

cv.waitKey(0)
```



## Switching Between Color Spaces

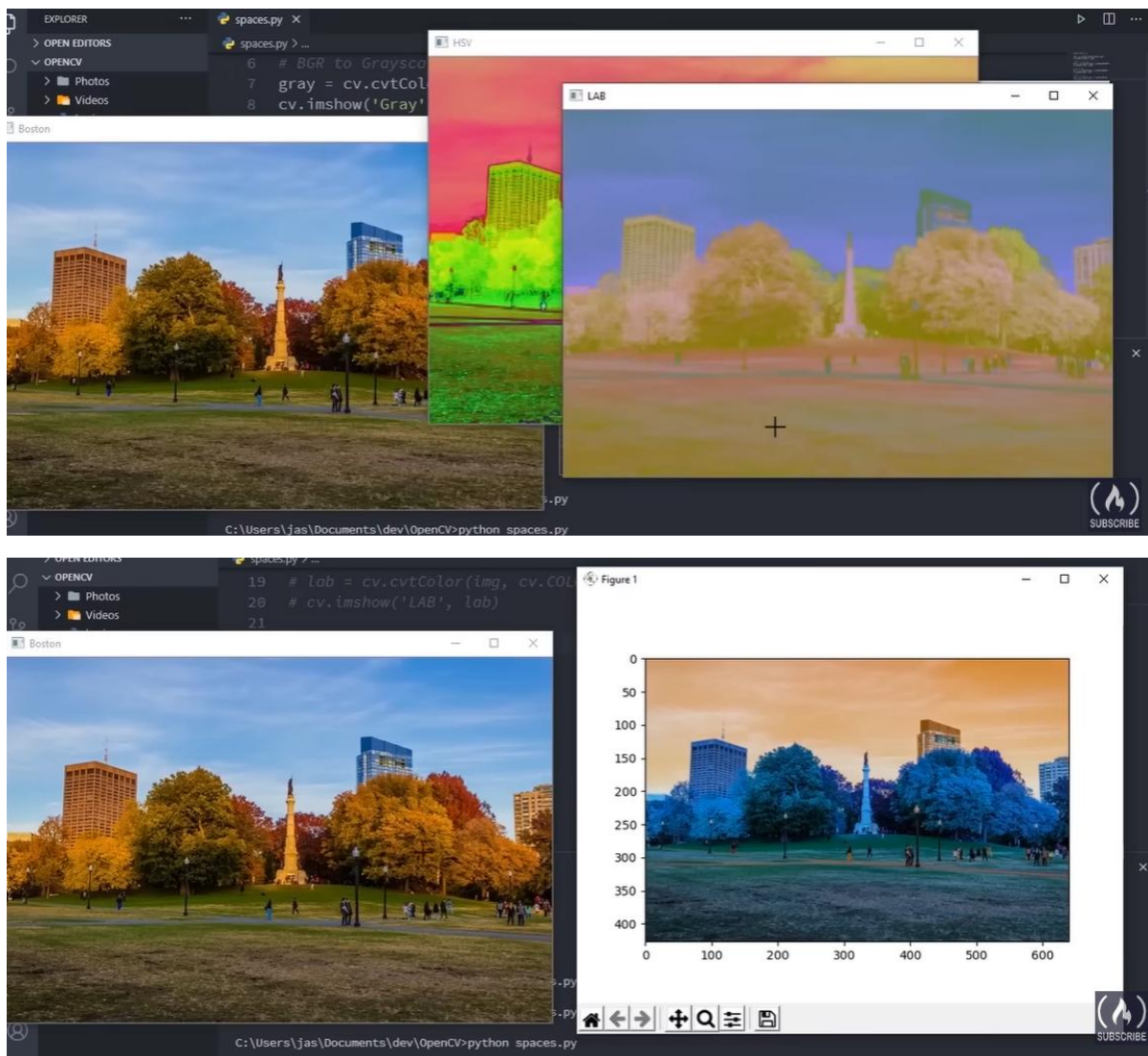
OpenCV lets you convert images between different color spaces using `cv2.cvtColor()`.

Supported conversions:

| From | To        |
|------|-----------|
| BGR  | Grayscale |
| BGR  | HSV       |
| BGR  | RGB       |
| BGR  | LAB       |

**Important:** You cannot convert directly from **Grayscale to HSV**. You must first convert **Gray → BGR**, then **BGR → HSV**.

**Note:** Images are read by OpenCV in BGR format. This is because when plotted using Matplotlib, the channels are treated in RGB order, leading to a reversal of the colors red and blue.



## Color Channels: Splitting and Merging

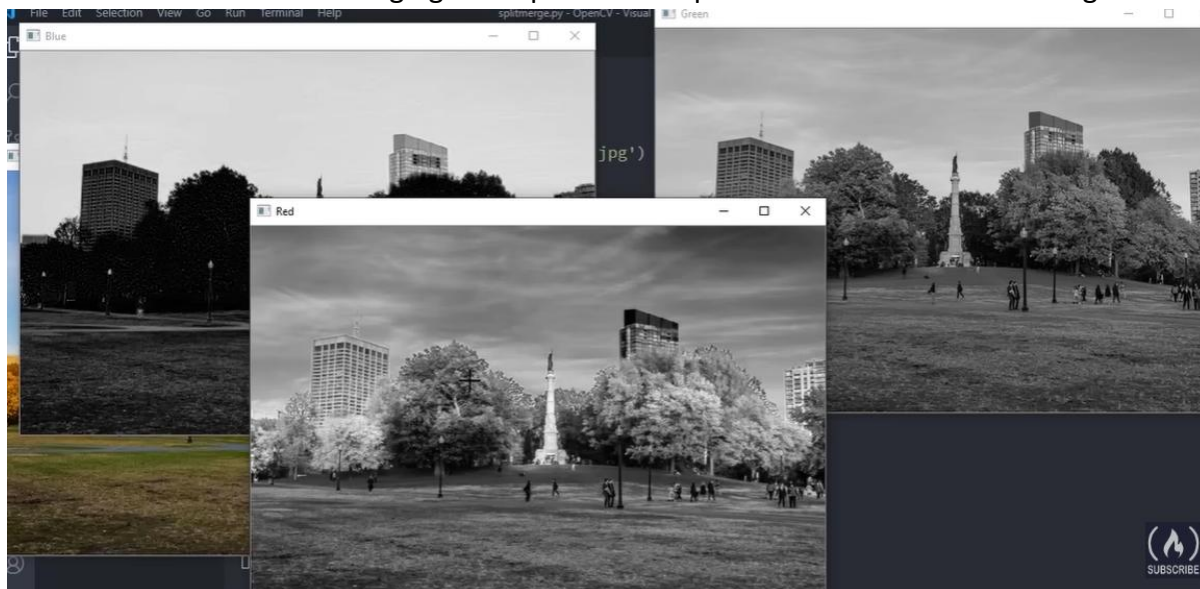
By using OpenCV, it becomes possible for you to separate BGR image into its three components separately i.e. blue, green, and red, through `cv2.split()` command.

When each component is separated and seen individually, it comes out to be a black and white image where:

- Lighter spots: high amount of that color in the pixel
- Darker spots: low amount of that color



The result obtained from merging the separated components is the same as the original BGR image.



```
import cv2 as cv
import numpy as np

img = cv.imread('Photos/boston.jpg')
cv.imshow('Boston', img)

b,g,r = cv.split(img)

cv.imshow('Blue', b)
cv.imshow('Green', g)
cv.imshow('Red', r)

print(img.shape)
print(b.shape)
```

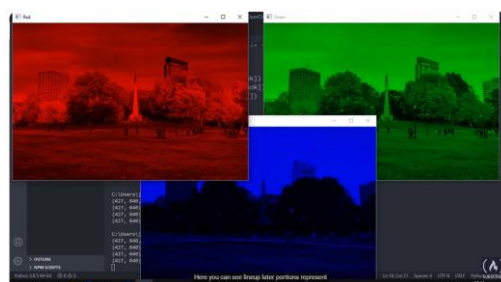
```
C:\Users\jas\Documents\dev\OpenCV>python splitmerge.py
(427, 640, 3)
(427, 640)
(427, 640)
(427, 640)
```

```
merged = cv.merge([b,g,r])
cv.imshow('Merged Image', merged)
```

```
b,g,r = cv.split(img)

blue = cv.merge([b,blank,blank])
green = cv.merge([blank,g,blank])
red = cv.merge([blank,blank,r])

cv.imshow('Blue', blue)
cv.imshow('Green', green)
cv.imshow('Red', red)
```



## Smoothing and Blurring (Noise Reduction)

Blurring replaces each pixel's value based on its surrounding pixels (the kernel). The blurring happens at the centre pixel of the kernel as a result of neighbouring pixel values.

| Method         | Description                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------------------|
| Average Blur   | Replaces center pixel with the average intensity of all surrounding pixels                                               |
| Gaussian Blur  | Similar to averaging but assigns <b>weighted values</b> to surrounding pixels — less blurring, more natural result       |
| Median Blur    | Replaces center pixel with the <b>median</b> intensity value of surrounding pixels — effective for salt-and-pepper noise |
| Bilateral Blur | Most effective — <b>blurs the image while preserving edges</b> . Great for denoising without losing important boundaries |

## Bitwise Operators (Used in Masks)

Bitwise operations work on pixel values at the binary level and are commonly used alongside masks:

- **AND** → keeps only pixels where both images are non-zero (used to apply a mask to an image)
- **OR** → keeps pixels where at least one image is non-zero
- **NOT** → **inverts the image** — black becomes white and white becomes black (inverts the mask)
- **XOR** → keeps pixels that differ between the two images

```

rectangle = cv.rectangle(blank.copy(), (30,30), (370,370), 255, -1)
circle = cv.circle(blank.copy(), (200,200), 200, 255, -1)

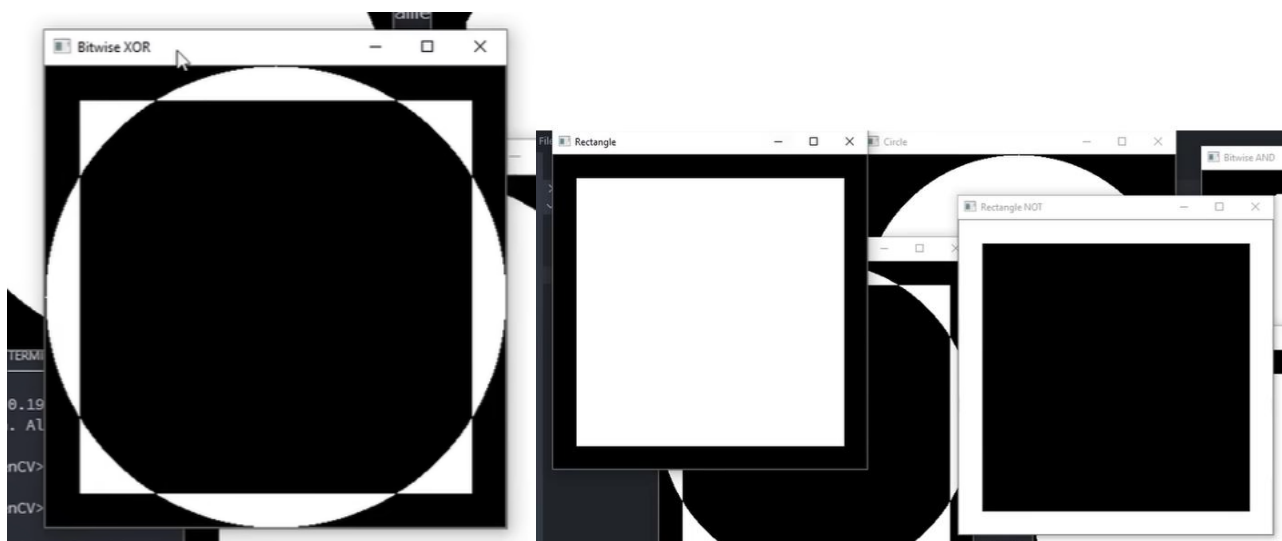
cv.imshow('Rectangle', rectangle)
cv.imshow('Circle', circle)

# bitwise AND --> intersecting regions
bitwise_and = cv.bitwise_and(rectangle, circle)
cv.imshow('Bitwise AND', bitwise_and)

# bitwise OR --> non-intersecting and intersecting regions
bitwise_or = cv.bitwise_or(rectangle, circle)
cv.imshow('Bitwise OR', bitwise_or)

# bitwise XOR --> non-intersecting regions
bitwise_xor = cv.bitwise_xor(rectangle, circle)
cv.imshow('Bitwise XOR', bitwise_xor)

```



## Masking (Focus on a Region of Interest)

Mask is a binary image (black and white), which can be used to extract a certain area from an image (for example, only faces).

Important things to keep in mind:

The size of the mask should be equal to the size of the image. Otherwise, it will not work.

One way to do this is: make a black empty image, add a white object (for example, a circle) in the area that needs to be highlighted, use `cv2.bitwise_and()` of the mask and the original image.

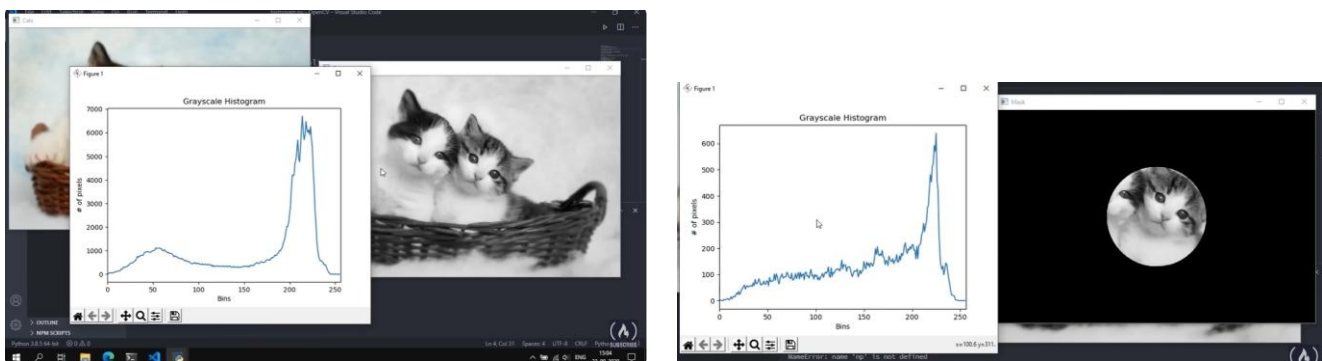
As a result, only the area covered by the mask remains white. All other areas become black.

---

## Computing Histograms

Histograms are useful in analyzing the intensity distribution in an image, meaning the number of pixels that occur at each level of intensity from 0 to 255 (black to white).

- The presence of a spike at around 225 indicates that the picture is mostly bright/white.
- Histograms can also be computed for the region inside a particular mask using `cv2.calcHist()`, instead of using `None`.



---

## Thresholding — Simple & Adaptive

**Simple Thresholding:** A manual threshold value is set. Pixels above it become white, below it become black. Includes an inverse mode where the result is flipped.

**Adaptive Thresholding:** The threshold is **calculated automatically** for different regions of the image — useful when lighting varies across the image.

- Uses a kernel (e.g., 11×11) and computes the **mean of surrounding pixels** to determine the optimal local threshold value.
  - No need to manually set a threshold — it adapts based on local context.
- 

## Gradients and Edge Detection (Beyond Canny)

### Laplacian Technique:

Gives effect like that of a sketch done in chalk or pencil; there will be a slightly blurred appearance. Uses second derivatives of intensity to detect edges.

### Sobel Technique:

- Sobel X → detects horizontal edges (vertical edges)
- Sobel Y → detects vertical edges (horizontal edges)
- Sobel X and Y together provide results similar to Laplacian technique

Laplacian vs Combined Sobel X & Y: Both techniques detect similar things but Laplacian is one single process while Sobel provides directionality

---

## Haar Cascades

The Haar Cascades is a pre-trained classifier technique used for object detection (initially used for face detection).

- Analyzes the features (edges and patterns) within an image to decide if a face (object) exists in the image
- Detects the number of faces in the image and draws boxes around them